
[L-1] VaultRewardDistributor credits emissions only to live shares while FirelightVault::payout slashes pending-withdrawal buckets, so exiting stakers bear loss with a structurally guaranteed zero return

Report: #90304 **Researcher:** @web3pavlou (Whitehat) **Program:** Audit Comp | Firelight **Submitted:** August 24, 2026 **Final status:** Closed by the project

Impact

Contract fails to deliver promised returns, but doesn't lose value.

Description

Brief / Intro

FirelightVault splits an incident loss across a wider set of capital than it splits emissions across.

Direction	Denominator
Loss — FirelightVault.payout	currentActiveAssets + withdrawAssets[cp+1] + withdrawAssets[cp+2]
Gain — VaultRewardDistributor.distribute*	totalAssets() only — pending withdrawals excluded

payout reaches into the withdrawal buckets and slashes them:

```
uint256 capturePeriodWithdrawals = withdrawAssets[capturePeriod + 1];
uint256 nextPeriodWithdrawals;
if (_currentPeriod == capturePeriod + 1) {
    nextPeriodWithdrawals = withdrawAssets[capturePeriod + 2];
}
uint256 activePayableAmount = currentActiveAssets + capturePeriodWithdrawals
↪ + nextPeriodWithdrawals;
```

```
...
withdrawAssets[capturePeriod + 1] = capturePeriodWithdrawals -
  ↪ paidFromCaptureWithdrawals;
```

Both distribution functions do only this:

```
$.vaultAsset.safeTransferFrom(msg.sender, address(_vault),
  ↪ vaultAssetAmount);
_vault.checkpointTotalAssets();
```

A raw balance increase raises `super.totalAssets()`, and the vault reports:

```
totalAssets() = super.totalAssets() - pendingWithdrawAssets
```

The entire emission therefore lands in the live-share pool. `withdrawAssets[*]` is never incremented by any distribution.

Root Cause

The root cause is an absent operation, and it is one grep:

```
grep -n "withdrawAssets\[\" contracts/core/FirelightVault.sol
```

Site	Operation
<code>_requestWithdraw</code>	<code>withdrawAssets[period] += assets</code> — the only increment in the contract
<code>payout</code>	<code>withdrawAssets[cp+1] = capturePeriodWithdrawals - paidFromCaptureWithdrawals</code> — decrement
<code>payout</code>	<code>withdrawAssets[cp+2] = nextPeriodWithdrawals - paidFromNextWithdrawals</code> — decrement
<code>claimWithdraw, withdrawalsOf</code>	Read only

There is exactly one `+=` and two `-=`. No distribution credits a bucket, and no recovery or admin path can restore one.

The single increase lives in `_requestWithdraw` and books a new claim with matching `withdrawShares`, so it raises the bucket's assets and shares together — it cannot restore value to a claim already sitting in it.

Vulnerability Details

The documented guarantee this makes unreachable

From Risk and Exposure, under “**What happens after a slash**”:

The staker’s vault share balance remains the same. The vault’s redemption value decreases to reflect the reduced backing. Future emission streams continue, rebuilding the redemption value over time.

For a slashed staker who has requested withdrawal, both load-bearing bullets are false.

Her shares were burned at request time (`_update(owner, address(0), shares)` in `_requestWithdraw`), so the first bullet does not hold.

Her claim now exists only as `withdrawAssets[P+1]`, which payout slashes and which nothing can restore, so the third bullet cannot hold either.

This is not an imperfect rebuild. PoC test 4 runs four consecutive emission cycles after the slash and her bucket returns the identical value every time — 3000000000000, five reads — while the staying cohort’s redemption value doubles from 200,000 to 400,000 FXRP over the same four distributions.

`VaultRewardDistributor` is the protocol’s only on-chain path for returning premium to stakers. The rebuild the documentation promises is delivered by these two functions or not at all.

Scope: this is not a privileged-actor report

The program excludes reports requiring:

a privileged role holder to act within the permissions explicitly granted to that role... unless the report demonstrates that the role can exceed its intended permissions or bypass an intended protocol constraint.

No role behaves incorrectly here. `DISTRIBUTOR_ROLE` forwards premium exactly as designed; `PAYOUT_ROLE` pays an approved claim exactly as designed.

The defect is in the accounting and fires identically on every distribution regardless of caller, timing, or intent. There is no adversary in this report and no operator error — the two functions cannot be called in a way that avoids it.

The intended constraint being broken is the vault’s own, stated in payout’s natspec:

Withdrawals requested during the capture period are assigned to `capturePeriod + 1` and remain payable while payout is allowed... otherwise exposed assets could escape by withdrawing first.

The protocol deliberately keeps exiting capital exposed to loss. Nothing anywhere states that the same capital should be excluded from the corresponding gain, and the loss path proves the protocol does not consider that capital to have left.

Sequence

No attacker. Every step is the protocol operating normally. Figures are from the PoC, FXRP at 6 decimals.

1. **Preconditions — internal:** at least one withdrawal request outstanding, which is the vault's normal steady state given redeem/withdraw burn shares immediately and freeze the claim for one to two periods. **External:** none.
2. **Period P opens.** Alice holds 600,000 FXRP of shares (60%), Bob 400,000 (40%). `totalAssetsAt(periodStart)` is 1,000,000 — both stakes form the base underwriting period P's cover.
3. **Alice exits.** She calls `redeem`. `_requestWithdraw` sets `withdrawAssets[P+1] = 600,000`, `pendingWithdrawAssets = 600,000`, and burns her shares. `totalAssets()` drops to 400,000. She cannot claim until period P+2 and remains fully slashable throughout.
4. **Period P's premium arrives.** `distributeRewards(50_000, ...)`. All 50,000 accrues to the 400,000 of live shares — a 12.5% return for Bob. Alice's bucket is untouched at 600,000. She earns 0.00% on capital that underwrote 60% of the period.
5. **An incident captured in period P is paid, 500,000.** Now the wider denominator applies:
 - $\text{paidFromActive} = 500,000 \times 450,000 / 1,050,000 = 214,285.714285$
 - $\text{paidFromCaptureWithdrawals} = 285,714.285715$
 - Alice's bucket drops to 314,285.714285.
6. **Period P+2.** Alice claims and receives 314,285.714285.

Impact Details

	Actual	Pro-rata	Delta
Alice —	314,285.714285	330,000	-15,714.285715
Bob —	235,714.285715	220,000	+15,714.285715

The two deltas are identical to the wei. 15,714.285715 FXP moved from the exiting cohort to the staying cohort, and the PoC asserts that equality rather than merely reporting it.

Victim class: every staker with a withdrawal request in flight, on every distribution, recurring each period.

The mirror is that the staying cohort earns a leveraged return — 100% of the premium on 40% of the capital that produced it.

Why “does not lose value” is the correct half of the impact string

The vault is exactly solvent throughout.

$$\begin{aligned} \text{aliceOut} + \text{bobOut} &= 550,000 \\ &= 1,000,000 + 50,000 - 500,000 \end{aligned}$$

This is asserted in the PoC.

Total obligations equal `super.totalAssets()` by construction, because:

$$\text{totalAssets}() = \text{super.totalAssets}() - \text{pendingWithdrawAssets}$$

and the two obligation terms sum back to the balance.

The protocol owes exactly what it holds. It assigned the wrong party.

That is what makes “**contract fails to deliver promised returns, but doesn’t lose value**” rather than insolvency: the shortfall is zero and the misallocation is total.

Severity

Low, matching the tier the program assigns to this impact.

The loss is real, realized, recurring and measurable, and it breaks a documented guarantee — but no funds are stolen, no funds are frozen, and the protocol remains solvent.

Two observations:

-
1. There is no attacker, and the impact string selected does not require one. “Contract fails to deliver promised returns” describes protocol behaviour, not an exploit. The behaviour is unconditional: it occurs on every distribution while any withdrawal is pending, which is the vault’s normal steady state.
 2. The withdrawal delay being intentional does not cover this. The delay explains why Alice cannot exit for two periods. It does not explain why she is slashed during those two periods and simultaneously excluded from the emissions earned in them. payout including her bucket is the protocol’s own statement that her capital is still at risk.

Distinction from Prior Coverage

Macro A-1 — Accepted Risks

New deposits can capture a pending reward distribution (accepted) — a deposit placed just before a distribution shares in yield it did not earn... It is mitigated by the withdrawal delay and by incident risk, since the new capital cannot exit immediately and remains exposed for the period.

That is the entry side: an incoming staker gains yield they did not earn.

This report is the exit side: a departing staker loses yield they did earn, while remaining liable for the loss.

Different cohort, opposite direction, and it appears nowhere in A-1’s issue list or its Trust Model.

Macro’s stated mitigation also rests on the exact principle this violates — the entry-side dilution is accepted because new capital **“remains exposed for the period.”**

Exiting capital is likewise exposed, by payout’s own reach into the buckets, and earns nothing.

Published Known Issues

The three listed entries cover:

1. The first-loss-buffer USD peg.
2. The first-loss buffer being an unescrowed external wallet.
3. Aggregate claims not being capped on-chain against total cover sold.

None concerns emission attribution or withdrawal-bucket accounting.

Design Notes

The scope page states:

FirelightVault intentionally deviates from standard ERC4626 withdrawal semantics. Withdraw and redeem create delayed withdrawal requests; users later call claimWithdraw.

This report does not dispute the delay — it concerns what happens to emissions and losses during it.

Recommended Mitigation

Route the distribution through a vault entrypoint rather than a raw transfer to address(vault), and split it using the denominator payout already computes:

```
// FirelightVault
function creditDistribution(uint256 amount) external
↳ onlyRole(CHECKPOINT_ROLE) {
    uint256 p = currentPeriod();
    uint256 wNext = withdrawAssets[p + 1];
    uint256 wAfter = withdrawAssets[p + 2];
    // `amount` has already landed in the balance by the time this is
↳ called, so remove it
    // from the denominator: the split must be against the pre-distribution
↳ base.
    uint256 base = totalAssets() - amount + wNext + wAfter;
    if (base == 0) return; // nothing outstanding to credit; leave the
↳ amount in totalAssets()
    uint256 toNext = amount.mulDiv(wNext, base);
    uint256 toAfter = amount.mulDiv(wAfter, base);
    withdrawAssets[p + 1] += toNext;
    withdrawAssets[p + 2] += toAfter;
    pendingWithdrawAssets += toNext + toAfter;
    // the residual stays in totalAssets() implicitly, as today
}
```

Then have distributeRewards / distributeIncentive call it after the transfer instead of only checkpointTotalAssets().

The distributor already holds CHECKPOINT_ROLE, so no new grant is needed.

checkpointTotalAssets() should run after the credit so the snapshot reflects the final split.

This mirrors the paidFromActive / paidFromCaptureWithdrawals / paidFromNextWithdrawals split payout performs, making the gain base identical to the loss base.

Alternative, Rejected

Removing pending withdrawals from payout's payable set restores symmetry from the other side but reopens the escape-by-withdrawing-first hole the natspec explicitly guards.

References

- `contracts/core/VaultRewardDistributor.sol` — `distributeRewards`, `distributeIncentive`: `transfer to address(_vault)` then `checkpointTotalAssets()`, with no bucket credit (root cause).
- `contracts/core/FirelightVault.sol` — `_requestWithdraw`: `withdrawAssets[period] += assets`, the only increment in the contract.
- `contracts/core/FirelightVault.sol` — `payout`: reads `withdrawAssets[cp+1]` / `[cp+2]` into `activePayableAmount`, then decrements both.
- `contracts/core/FirelightVault.sol` — `totalAssets()` = `super.totalAssets()` - `pendingWithdrawAssets`.
- `contracts/core/FirelightVault.sol` — `claimWithdraw`: converts against the frozen bucket totals.
- Risk and Exposure — “What happens after a slash”.

Proof of Concept

Foundry, against the real `FirelightVault` and the real `VaultRewardDistributor`. No allocator and no `IncidentManager` — `payout` is called directly under `PAYOUT_ROLE`, which produces vault state identical to the one `IncidentManager._executePayout` produces.

Four tests:

1. The finding.
2. A control isolating the cause from the no-withdrawal side.
3. A control isolating the cause from the no-distribution side.
4. A direct demonstration that the documented rebuild is unreachable.

Note on `via_ir`

`foundry.toml` sets `via_ir = true`, and the IR optimizer `common-subexpression-eliminates` `TIMESTAMP` within a function body, so every warp below is derived from an absolute anchor read fresh from the vault.

The same rule forces `balanceOf` to be cached before `vm.prank`, since Solidity evaluates arguments before the call and would otherwise consume the prank.

test/Foundry/Base.t.sol

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;
import {Test, console2} from "forge-std/Test.sol";
import {ERC1967Proxy} from
    ↪ "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {FirelightVault} from "contracts/core/FirelightVault.sol";
import {CoverOrderAllocator} from "contracts/core/CoverOrderAllocator.sol";
import {CoverNFT} from "contracts/core/CoverNFT.sol";
import {IncidentManager} from "contracts/core/IncidentManager.sol";
import {VaultRewardDistributor} from
    ↪ "contracts/core/VaultRewardDistributor.sol";
import {ICoverOrderAllocator} from
    ↪ "contracts/core/interfaces/ICoverOrderAllocator.sol";
import {IFirelightVault} from
    ↪ "contracts/core/interfaces/IFirelightVault.sol";
import {IAggregatorV3} from "contracts/core/interfaces/IAggregatorV3.sol";
import {MockFAsset} from "./mocks/MockFAsset.sol";
import {MockERC20} from "contracts/test/MockERC20.sol";
import {MockAggregatorV3} from "contracts/test/MockAggregatorV3.sol";
import {MockCoverOrderAllocatorVault} from
    ↪ "contracts/test/MockCoverOrderAllocatorVault.sol";
import {IIIncidentManager} from
    ↪ "contracts/core/interfaces/IIIncidentManager.sol";
/**
 * @title Base
 * @notice Solidity port of test/setup/fixtures.js – `deployVault` and
 *         `deployCoverOrderAllocator` – so the Hardhat suite and the
    ↪ Foundry suite
 *         exercise identical deployments.
 * Two deliberate deviations from the JS, both noted where they matter:
 * 1. Proxies are ERC1967Proxy rather than the Transparent proxies
    ↪ `upgrades.deployProxy`
 *     creates. Storage layout and delegatecall semantics are identical, and
    ↪ it removes the
 *     ProxyAdmin indirection that would otherwise make every `vm.prank`
    ↪ two-hop. Use
```

```

*      `deployVaultTransparent()` when the proxy *type* is the thing under
↪ test
*      (e.g. the unguarded `initializeV2` reinitializer).
*      2. The underlying is `MockFAsset`, not the real `deployFAsset` helper –
↪ `contracts/fasset`
*      is pragma 0.8.23 and skipped in foundry.toml. MockFAsset reproduces
*      `FAsset.transferFrom` line for line, including the round-UP fee, and
↪ defaults to a
*      zero fee so it behaves exactly like a plain ERC20 until a test opts in.
*/
abstract contract Base is Test {
    // --- config (mirrors DEFAULT_CONFIG) ---
    uint8 internal constant DECIMALS = 6;
    string internal constant UNDERLYING = "fXRP";
    string internal constant LST = "stfXRP";
    uint256 internal constant INITIAL_DEPOSIT_LIMIT = 50_000_000_000;
    uint48 internal constant PERIOD_DURATION = 604800;
    // --- constants (mirrors the JS constants block) ---
    uint256 internal constant BPS = 10_000;
    uint256 internal constant LEV = 10_000;
    uint256 internal constant SECS_PER_YEAR = 365 days;
    string internal constant PROTOCOL_MORPHO = "Morpho";
    string internal constant PROTOCOL_AAVE = "Aave";
    bytes32 internal constant MARKET_A = bytes32("market-A");
    bytes32 internal constant MARKET_B = bytes32("market-B");
    // --- vault fixture ---
    MockFAsset internal token;
    FirelightVault internal vault;
    address internal deployer;
    address internal rescuer;
    address internal blocklister;
    address internal pauser;
    address internal limitUpdater;
    address internal periodConfigurationUpdater;
    address internal payoutSigner;
    address internal payoutAllowlister;
    address internal payoutReceiver;
    address internal incidentRole;
    address internal checkpointer;
    address[3] internal users;
    VaultRewardDistributor internal distributor;
    address internal distributorEOA;
    // --- allocator fixture ---

```

```

CoverOrderAllocator internal allocator;
CoverNFT internal coverNFT;
MockCoverOrderAllocatorVault internal mockVault;
MockAggregatorV3 internal priceFeed;
MockERC20 internal usdc;
MockERC20 internal usdt;
address internal admin;
address internal curator;
address internal allocatorRole;
address internal configAdmin;
address internal adminRole;
address internal premiumCollector;
address internal firstLossBufferWallet;
address internal buyer1;
address internal buyer2;
address internal beneficiary;
bytes32 internal marketIdA;
bytes32 internal marketIdB;
bytes32 internal concHashMorpho;
bytes32 internal concHashAave;
// --- incident manager fixture ---
IncidentManager internal incidentManager;
address internal imAdmin;
address internal imCurator;
address internal imApprover;
address internal imRejecter;
address internal imInvalidator;
address internal imConfigAdmin;
address internal imPayoutAdmin;
address internal imPriceFeedAdmin;
address internal imPayoutReceiver;
// =====
// deployVault (port of fixtures.js::deployVault)
// =====
function deployVault() internal {
    if (block.timestamp < 1_700_000_000) vm.warp(1_700_000_000);
    deployer = _signer("deployer");
    rescuer = _signer("rescuer");
    blocklister = _signer("blocklister");
    pauser = _signer("pauser");
    limitUpdater = _signer("limitUpdater");
    periodConfigurationUpdater = _signer("periodConfigurationUpdater");
    users[0] = _signer("user1");

```

```

users[1] = _signer("user2");
users[2] = _signer("user3");
token = new MockFAsset(UNDERLYING, UNDERLYING, DECIMALS);
FirelightVault.InitParams memory p = FirelightVault.InitParams({
    defaultAdmin: deployer,
    limitUpdater: limitUpdater,
    blocklister: blocklister,
    pauser: pauser,
    periodConfigurationUpdater: periodConfigurationUpdater,
    rescuer: rescuer,
    depositLimit: INITIAL_DEPOSIT_LIMIT,
    periodConfigurationDuration: PERIOD_DURATION
});
FirelightVault impl = new FirelightVault();
ERC1967Proxy proxy = new ERC1967Proxy(
    address(impl), abi.encodeCall(FirelightVault.initialize,
↪ (IERC20(address(token)), LST, LST, abi.encode(p)))
);
vault = FirelightVault(address(proxy));
payoutSigner = _signer("payoutSigner");
payoutAllowlister = _signer("payoutAllowlister");
payoutReceiver = _signer("payoutReceiver");
incidentRole = _signer("incident");
checkpointer = _signer("checkpointer");
vm.startPrank(deployer);
vault.grantRole(vault.PAYOUT_ROLE(), payoutSigner);
vault.grantRole(vault.PAYOUT_ALLOWLIST_ROLE(), payoutAllowlister);
vault.grantRole(vault.INCIDENT_ROLE(), incidentRole);
vault.grantRole(vault.CHECKPOINT_ROLE(), checkpointer);
vm.stopPrank();
vm.prank(payoutAllowlister);
vault.addToPayoutAllowlist(payoutReceiver);
}
// =====
// deployCoverOrderAllocator (port of
↪ fixtures.js::deployCoverOrderAllocator)
// =====
function deployCoverOrderAllocator() internal {
    if (block.timestamp < 1_700_000_000) vm.warp(1_700_000_000);
    admin = _signer("admin");
    curator = _signer("curator");
    allocatorRole = _signer("allocatorRole");
    configAdmin = _signer("configAdmin");

```

```

    adminRole = _signer("adminRole");
    premiumCollector = _signer("premiumCollector");
    firstLossBufferWallet = _signer("firstLossBufferWallet");
    buyer1 = _signer("buyer1");
    buyer2 = _signer("buyer2");
    beneficiary = _signer("beneficiary");
    mockVault = new MockCoverOrderAllocatorVault();
    mockVault.setCurrentPeriod(1);
    mockVault.setTotalAssets(0);
    for (uint256 i = 1; i <= 3; i++) {
        mockVault.setPeriodConfiguration(
            i, IFirelightVault.PeriodConfiguration({epoch: 0, duration:
↪ PERIOD_DURATION, startingPeriod: i})
        );
    }
    usdc = new MockERC20("MockUSDC", "mUSDC", 6);
    usdt = new MockERC20("MockUSDT", "mUSDT", 6);
    mockVault.setAsset(address(usdc));
    priceFeed = new MockAggregatorV3(18, int256(10 ** 18));
    CoverNFT nftImpl = new CoverNFT();
    coverNFT = CoverNFT(
        address(
            new ERC1967Proxy(
                address(nftImpl),
                abi.encodeCall(
                    CoverNFT.initialize,
                    ("Firelight Cover", "FLCOVER", "", admin,
↪ address(this), address(0), address(0))
                )
            )
        )
    );
    ICoverOrderAllocator.Market[] memory markets = new
↪ ICoverOrderAllocator.Market[](2);
    markets[0] = ICoverOrderAllocator.Market({chainId: 1, protocol:
↪ PROTOCOL_MORPHO, market: MARKET_A});
    markets[1] = ICoverOrderAllocator.Market({chainId: 10, protocol:
↪ PROTOCOL_AAVE, market: MARKET_B});
    ICoverOrderAllocator.ProtocolConcentrationInput[] memory concs =
        new ICoverOrderAllocator.ProtocolConcentrationInput[](2);
    concs[0] = ICoverOrderAllocator.ProtocolConcentrationInput({
        chainId: 1, protocol: PROTOCOL_MORPHO,
↪ maxProtocolConcentrationBps: 6000

```

```

    });
    concs[1] = ICoverOrderAllocator.ProtocolConcentrationInput({
        chainId: 10, protocol: PROTOCOL_AAVE,
↪ maxProtocolConcentrationBps: 4000
    });
    address[] memory premiumTokens = new address[](2);
    premiumTokens[0] = address(usdc);
    premiumTokens[1] = address(usdt);
    ICoverOrderAllocator.InitParams memory ip =
↪ ICoverOrderAllocator.InitParams({
        vault: IFirelightVault(address(mockVault)),
        premiumCollector: premiumCollector,
        coverNFT: coverNFT,
        priceFeedAdapter: IAggregatorV3(address(priceFeed)),
        maxPriceAge: 7 days,
        premiumTokens: premiumTokens,
        admin: admin,
        adminRole: adminRole,
        curatorRole: curator,
        allocatorRole: allocatorRole,
        configAdminRole: configAdmin,
        initialProtocolConcentrations: concs,
        newMarkets: markets,
        capacityConfig: ICoverOrderAllocator.CapacityConfig({
            minCAR: 12000,
            firstLossBufferToken: IERC20(address(usdc)),
            firstLossBuffer: firstLossBufferWallet,
            effectiveLeverage: 24000,
            minOrderMarketCoverAmount: 1,
            divergenceToleranceBps: 0
        })
    });
    CoverOrderAllocator aImpl = new CoverOrderAllocator();
    allocator = CoverOrderAllocator(
        address(new ERC1967Proxy(address(aImpl),
↪ abi.encodeCall(CoverOrderAllocator.initialize, (ip))))
    );
    bytes32 minterRole = coverNFT.MINTER_ROLE();
    vm.prank(admin);
    coverNFT.grantRole(minterRole, address(allocator));
    marketIdA = computeMarketId(1, PROTOCOL_MORPHO, MARKET_A);
    marketIdB = computeMarketId(10, PROTOCOL_AAVE, MARKET_B);

```

```

        concHashMorpho = computeProtocolConcentrationHash(1,
↪  PROTOCOL_MORPHO);
        concHashAave = computeProtocolConcentrationHash(10, PROTOCOL_AAVE);
    }
    // =====
    // deployFullStack – allocator wired to the REAL vault
    // =====
    /**
     * @dev `deployCoverOrderAllocator()` points the allocator at
↪  `MockCoverOrderAllocatorVault`,
     *      which returns hardcoded values and cannot execute a real
↪  `payout`. Anything that
     *      tests the *interaction* between vault state and allocator
↪  capacity needs the real
     *      vault instead.
     *      `FirelightVault` does not inherit `IFirelightVault`, but every
↪  selector the allocator
     *      calls matches, so the interface cast resolves at runtime. (Struct
↪  nominal typing only
     *      binds at compile time; through an interface it is a plain ABI
↪  call.)
     *      Capacity is deliberately isolated to the vault leg: the
↪  first-loss buffer wallet is
     *      left at zero balance, so `_computeAvailableCapacity` reduces to
     *      `totalAssetsAt(periodStart) * 1e12 * effectiveLeverage / minCAR`.
     *      Call after `deployVault()`.
    */
    function deployFullStack() internal {
        admin = _signer("admin");
        curator = _signer("curator");
        allocatorRole = _signer("allocatorRole");
        configAdmin = _signer("configAdmin");
        adminRole = _signer("adminRole");
        premiumCollector = _signer("premiumCollector");
        firstLossBufferWallet = _signer("firstLossBufferWallet");
        buyer1 = _signer("buyer1");
        buyer2 = _signer("buyer2");
        beneficiary = _signer("beneficiary");
        usdc = new MockERC20("MockUSDC", "mUSDC", 6);
        priceFeed = new MockAggregatorV3(18, int256(10 ** 18));
        CoverNFT nftImpl = new CoverNFT();
        coverNFT = CoverNFT(
            address(

```

```

        new ERC1967Proxy(
            address(nftImpl),
            abi.encodeCall(
                CoverNFT.initialize,
                ("Firelight Cover", "FLCOVER", "", admin,
↪ address(this), address(0), address(0))
            )
        )
    );
    ICoverOrderAllocator.Market[] memory markets = new
↪ ICoverOrderAllocator.Market[](1);
    markets[0] = ICoverOrderAllocator.Market({chainId: 1, protocol:
↪ PROTOCOL_MORPHO, market: MARKET_A});
    ICoverOrderAllocator.ProtocolConcentrationInput[] memory concs =
        new ICoverOrderAllocator.ProtocolConcentrationInput[](1);
    concs[0] = ICoverOrderAllocator.ProtocolConcentrationInput({
        chainId: 1, protocol: PROTOCOL_MORPHO,
↪ maxProtocolConcentrationBps: 10000
    });
    address[] memory premiumTokens = new address[](1);
    premiumTokens[0] = address(usdc);
    ICoverOrderAllocator.InitParams memory ip =
↪ ICoverOrderAllocator.InitParams({
        vault: IFirelightVault(address(vault)),
        premiumCollector: premiumCollector,
        coverNFT: coverNFT,
        priceFeedAdapter: IAggregatorV3(address(priceFeed)),
        maxPriceAge: 7 days,
        premiumTokens: premiumTokens,
        admin: admin,
        adminRole: adminRole,
        curatorRole: curator,
        allocatorRole: allocatorRole,
        configAdminRole: configAdmin,
        initialProtocolConcentrations: concs,
        newMarkets: markets,
        capacityConfig: ICoverOrderAllocator.CapacityConfig({
            minCAR: 12000,
            firstLossBufferToken: IERC20(address(usdc)),
            firstLossBuffer: firstLossBufferWallet,
            effectiveLeverage: 24000,
            minOrderMarketCoverAmount: 1,

```

```

        divergenceToleranceBps: 0
    })
});
CoverOrderAllocator aImpl = new CoverOrderAllocator();
allocator = CoverOrderAllocator(
    address(new ERC1967Proxy(address(aImpl),
↪ abi.encodeCall(CoverOrderAllocator.initialize, (ip))))
);
bytes32 minterRole = coverNFT.MINTER_ROLE();
vm.prank(admin);
coverNFT.grantRole(minterRole, address(allocator));
marketIdA = computeMarketId(1, PROTOCOL_MORPHO, MARKET_A);
conchHashMorpho = computeProtocolConcentrationHash(1,
↪ PROTOCOL_MORPHO);
}
/**
 * @dev Wires the real IncidentManager to the real allocator and the
↪ real vault.
 *      `initialize` derives its vault from `allocator.vault()`, so this
↪ MUST be called
 *      after `deployFullStack()` – after `deployCoverOrderAllocator()`
↪ it would bind to
 *      the mock vault and `payout` would be unreachable.
 *      The manager needs two roles on the vault: INCIDENT_ROLE (so
↪ `_increaseIncident`
 *      can set the deposit-block flag) and PAYOUT_ROLE (so
↪ `_executePayout` can draw).
 *      Its payout receiver must also be on the vault's payout allowlist.
 *      The first-loss buffer wallet is left at zero balance by
↪ `deployFullStack()`, so
 *      `_executePayout` skips the buffer leg entirely and bills the
↪ vault for the whole
 *      assessed loss – which is what isolates the vault-side effect.
 */
function deployIncidentManager() internal {
    imAdmin = _signer("imAdmin");
    imCurator = _signer("imCurator");
    imApprover = _signer("imApprover");
    imRejecter = _signer("imRejecter");
    imInvalidator = _signer("imInvalidator");
    imConfigAdmin = _signer("imConfigAdmin");
    imPayoutAdmin = _signer("imPayoutAdmin");
    imPriceFeedAdmin = _signer("imPriceFeedAdmin");

```

```

imPayoutReceiver = _signer("imPayoutReceiver");
IncidentManager imImpl = new IncidentManager();
incidentManager = IncidentManager(
    address(
        new ERC1967Proxy(
            address(imImpl),
            abi.encodeCall(
                IncidentManager.initialize,
                (
                    imAdmin,
                    imCurator,
                    imApprover,
                    imRejecter,
                    imInvalidator,
                    imConfigAdmin,
                    imPayoutAdmin,
                    imPriceFeedAdmin,
                    imPayoutReceiver,
                    ICoverOrderAllocator(address(allocator)),
                    IAggregatorV3(address(priceFeed)),
                    7 days
                )
            )
        )
    )
);
bytes32 incidentR = vault.INCIDENT_ROLE();
bytes32 payoutR = vault.PAYOUT_ROLE();
vm.startPrank(deployer);
vault.grantRole(incidentR, address(incidentManager));
vault.grantRole(payoutR, address(incidentManager));
vm.stopPrank();
vm.prank(payoutAllowlister);
vault.addToPayoutAllowlist(imPayoutReceiver);
}
/// @dev MockAggregatorV3 pins `updatedAt` at construction, so any
↪ `vm.warp` past
///     `maxPriceAge` (7 days here) makes `PriceFeed.getPrice` revert
↪ `PriceFeedTooOld`.
///     Call before any commit/recommit that follows a warp.
function refreshPriceFeed() internal {
    priceFeed.setUpdatedAt(block.timestamp);
}

```

```

    /// @dev Capacity the allocator will compute, replicating
↪  `_computeAvailableCapacity`
    ///      with a zero first-loss buffer and a $1.00 / 18-decimal feed.
    function expectedCapacity(uint256 vaultAssets6dp) internal pure returns
↪  (uint256) {
        return (vaultAssets6dp * 10 ** 12) * 24000 / 12000;
    }
    // =====
    // Helpers (ports of the JS utility closures)
    // =====
    function mintAndApprove(uint256 amount, address user) internal {
        token.mintTo(user, amount);
        vm.prank(user);
        token.approve(address(vault), amount);
    }
    /**
     * @dev Wires the real `VaultRewardDistributor` – the protocol's ONLY
↪  on-chain path for
     *      returning premium to stakers. Premiums are collected in a premium
↪  token, swapped
     *      off-chain, and pushed back in as vault asset; the distributor
↪  then calls
     *      `checkpointTotalAssets()`, which is why the DISTRIBUTOR CONTRACT
↪  (not the EOA)
     *      needs CHECKPOINT_ROLE on the vault.
     *      `distributeRewards` pulls with `safeTransferFrom(msg.sender,
↪  vault, amount)`, so
     *      the EOA holding DISTRIBUTOR_ROLE must hold the asset and approve
↪  the distributor.
     *      Call after `deployVault()`.
     */
    function deployRewardDistributor() internal {
        distributorEOA = _signer("distributorEOA");
        VaultRewardDistributor dImpl = new VaultRewardDistributor();
        distributor = VaultRewardDistributor(
            address(
                new ERC1967Proxy(
                    address(dImpl),
                    abi.encodeCall(
                        VaultRewardDistributor.initialize,
                        (IFirelightVault(address(vault)), deployer,
↪  distributorEOA, address(0))
                    )
            )

```

```

        )
    )
);
bytes32 checkpointRole = vault.CHECKPOINT_ROLE();
vm.prank(deployer);
vault.grantRole(checkpointRole, address(distributor));
}
/// @dev Push `amount` of vault asset in as period premium, exactly as
↪ production does.
function distributePremium(uint256 amount) internal {
    token.mintTo(distributorEOA, amount);
    vm.startPrank(distributorEOA);
    token.approve(address(distributor), amount);
    distributor.distributeRewards(amount, address(token), amount,
↪ uint48(block.timestamp));
    vm.stopPrank();
}
/**
 * @dev The fixture keeps `fixtures.js`'s 50,000-token deposit limit
↪ (5e10 at 6 decimals).
 *     That is small enough that most PoCs trip `DepositLimitExceeded`
↪ before reaching
 *     the behaviour under test – `_depositFunds` checks `totalAssets()
↪ + assets > depositLimit`.
 *     Raise it through the real `DEPOSIT_LIMIT_UPDATE_ROLE` rather
↪ than editing the fixture,
 *     so the deployment stays identical to the Hardhat suite.
 *     1e15 == 1 billion tokens: ample headroom, and small enough that
 *     `convertToShares(depositLimit)` inside `maxMint` cannot overflow
↪ `mulDiv`.
 */
function raiseDepositLimit() internal {
    raiseDepositLimit(1_000_000_000e6);
}
function raiseDepositLimit(uint256 newLimit) internal {
    vm.prank(limitUpdater);
    vault.updateDepositLimit(newLimit);
}
function fundAndApprove(address buyer, MockERC20 t, uint256 amount)
↪ internal {
    t.mint(buyer, amount);
    vm.prank(buyer);
    t.approve(address(allocator), amount);

```

```

    }
    function advanceToPeriod(uint256 p) internal {
        mockVault.setCurrentPeriod(p);
    }
    function warpPeriods(uint256 n) internal {
        vm.warp(block.timestamp + n * PERIOD_DURATION);
    }
    /**
     * @dev Timestamp at which `period` begins, for the real vault.
     * NOTE the type. `PeriodConfiguration` is declared TWICE in this
    ↪ codebase –
     * `FirelightVaultStorage.sol:51` and `IFirelightVault.sol:33` – and
    ↪ they are distinct
     * nominal types. `FirelightVault` inherits FirelightVaultStorage and
    ↪ does NOT implement
     * IFirelightVault, so this getter returns the STORAGE struct. Using the
    ↪ interface struct
     * here fails with "not implicitly convertible".
     * Conversely `MockCoverOrderAllocatorVault` is IFirelightVault`, so its
     * `setPeriodConfiguration` takes the INTERFACE struct. Both spellings
    ↪ are correct in
     * their own place; do not unify them.
     * For the current period prefer `vault.currentPeriodStart()`, which
    ↪ needs no struct.
     */
    function periodStart(uint256 period) internal view returns (uint48) {
        FirelightVault.PeriodConfiguration memory c =
    ↪ vault.periodConfigurationAtNumber(period);
        return uint48(c.epoch + (period - c.startingPeriod) * c.duration);
    }
    function computeMarketId(uint64 chainId, string memory protocol, bytes32
    ↪ market) internal pure returns (bytes32) {
        return keccak256(abi.encode(chainId, protocol, market));
    }
    function computeProtocolConcentrationHash(uint64 chainId, string memory
    ↪ protocol) internal pure returns (bytes32) {
        return keccak256(abi.encode(chainId, protocol));
    }
    /**
     * @dev Leaf encoding used by `_settleCoverOrder`: OZ StandardMerkleTree
    ↪ double-hashing.
     * For a SINGLE-order tree the root IS the leaf and the proof is
    ↪ empty, which is all

```

```

    *      most PoCs need. Multi-order trees need a real tree builder.
    */
function settleLeaf(uint256 orderId,
↪ ICoverOrderAllocator.MarketCoverAllocation[] memory allocs)
    internal
    pure
    returns (bytes32)
{
    return keccak256(bytes.concat(keccak256(abi.encode(orderId,
↪ allocs))));
}
function merkleRoot2(bytes32 a, bytes32 b) internal pure returns
↪ (bytes32) {
    return a < b ? keccak256(abi.encode(a, b)) : keccak256(abi.encode(b,
↪ a));
}
/**
 * @dev Funded EOA, replacing the JS `randomSigner()`.
 */
function _signer(string memory label) internal returns (address a) {
    a = makeAddr(label);
    vm.deal(a, 1000 ether);
}
function oneMarket(bytes32 marketId, uint256 amount)
    internal
    pure
    returns (ICoverOrderAllocator.MarketCoverAllocation[] memory arr)
{
    arr = new ICoverOrderAllocator.MarketCoverAllocation[](1);
    arr[0] = ICoverOrderAllocator.MarketCoverAllocation({marketId:
↪ marketId, allocatedCover: amount});
}
function oneMarketInput(bytes32 marketId, uint32 rateAnnual, uint256
↪ amount)
    internal
    pure
    returns (ICoverOrderAllocator.MarketAllocationInput[] memory arr)
{
    arr = new ICoverOrderAllocator.MarketAllocationInput[](1);
    arr[0] = ICoverOrderAllocator.MarketAllocationInput({
        marketId: marketId, coverRateAnnual: rateAnnual, coverAmount:
↪ amount
    });
}

```

```
}  
}
```

test/Foundry/mocks/MockFAsset.sol

```
// SPDX-License-Identifier: MIT  
pragma solidity 0.8.28;  
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";  
import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";  
/**  
 * @title MockFAsset  
 * @notice Faithful replica of the Flare FAsset transfer semantics used by  
 * ↪ FXRP, the  
 * ↪ FirelightVault's underlying asset.  
 * Mirrors contracts/fasset/fassetToken/implementation/FAsset.sol:  
 * ↪ uint256 transferFee = _transferFeeAmount(_amount);  
 * ↪ _transfer(_from, _to, _amount - transferFee);  
 * ↪ _payTransferFee(_from, transferFee);  
 * So `transferFrom(from, to, amount)` debits `from` by exactly `amount` and  
 * ↪ credits  
 * ↪ `to` with `amount - fee`. The fee is rounded UP, which is why a 1-unit  
 * ↪ transfer at  
 * ↪ any non-zero rate delivers zero.  
 * `transferFeeMillionths` is governance-settable on the real FAsset via  
 * `ITransferFees.setTransferFeeMillionths`, so tests parameterise it.  
 * ↪ Default 0 keeps  
 * ↪ every existing test green; set it non-zero to exercise the  
 * ↪ fee-on-transfer findings.  
 */  
contract MockFAsset is ERC20 {  
    uint8 private immutable _decimals;  
    uint256 public transferFeeMillionths;  
    address public feeCollector;  
    constructor(string memory n, string memory s, uint8 d) ERC20(n, s) {  
        _decimals = d;  
        feeCollector = address(0xFEE);  
    }  
    function decimals() public view override returns (uint8) {  
        return _decimals;  
    }  
    function setTransferFeeMillionths(uint256 v) external {  
        require(v < 1_000_000, "fee >= 100%");  
    }  
}
```

```

        transferFeeMillionths = v;
    }
    function mint(address to, uint256 amount) external {
        _mint(to, amount);
    }
    function mintTo(address to, uint256 amount) external {
        _mint(to, amount);
    }
    function _transferFeeAmount(uint256 amount) public view returns
    ↪ (uint256) {
        if (transferFeeMillionths == 0) return 0;
        return Math.mulDiv(amount, transferFeeMillionths, 1_000_000,
    ↪ Math.Rounding.Ceil);
    }
    function transfer(address to, uint256 amount) public override returns
    ↪ (bool) {
        uint256 fee = _transferFeeAmount(amount);
        _transfer(_msgSender(), to, amount - fee);
        if (fee > 0) _transfer(_msgSender(), feeCollector, fee);
        return true;
    }
    function transferFrom(address from, address to, uint256 amount) public
    ↪ override returns (bool) {
        _spendAllowance(from, _msgSender(), amount);
        uint256 fee = _transferFeeAmount(amount);
        _transfer(from, to, amount - fee);
        if (fee > 0) _transfer(from, feeCollector, fee);
        return true;
    }
}

```

test/Foundry/DistributionCohortAsymmetry.t.sol

```

// SPDX-License-Identifier: MIT
pragma solidity 0.8.28;
import {Base} from "../Base.t.sol";
import {console2} from "forge-std/Test.sol";
import {FirelightVault} from "contracts/core/FirelightVault.sol";
contract DistributionCohortAsymmetry is Base {
    uint256 constant ALICE_STAKE = 600_000e6;
    uint256 constant BOB_STAKE = 400_000e6;
    uint256 constant PREMIUM = 50_000e6;
}

```

```

uint256 constant LOSS = 500_000e6;
address alice;
address bob;
function setUp() public {
    deployVault();
    deployRewardDistributor();
    raiseDepositLimit();
    alice = users[0];
    bob = users[1];
}
function _stakeAndEnterPeriodP() internal returns (uint256 P, uint48
↪ pStart) {
    mintAndApprove(ALICE_STAKE, alice);
    vm.prank(alice);
    vault.deposit(ALICE_STAKE, alice);
    mintAndApprove(BOB_STAKE, bob);
    vm.prank(bob);
    vault.deposit(BOB_STAKE, bob);
    vm.warp(uint256(vault.currentPeriodStart()) + PERIOD_DURATION + 1
↪ hours);
    P = vault.currentPeriod();
    pStart = vault.currentPeriodStart();
    assertEq(
        vault.totalAssetsAt(pStart),
        ALICE_STAKE + BOB_STAKE,
        "both stakes are in the period-start snapshot that sets the
↪ exposure cap"
    );
}
function testPendingWithdrawalBearsTheSlashButEarnsNoneOfThePremium()
↪ public {
    (uint256 P, uint48 pStart) = _stakeAndEnterPeriodP();
    uint256 bucket = P + 1;
    uint256 aliceShares = vault.balanceOf(alice);
    vm.prank(alice);
    vault.redeem(aliceShares, alice, alice);
    assertEq(vault.withdrawAssets(bucket), ALICE_STAKE, "Alice's claim
↪ booked into P+1");
    assertEq(vault.balanceOf(alice), 0, "shares burned at request time");
    assertEq(vault.totalAssets(), BOB_STAKE, "totalAssets() now excludes
↪ Alice");
    distributePremium(PREMIUM);
    assertEq(

```

```

        vault.withdrawAssets(bucket), ALICE_STAKE,
        "Alice's bucket is untouched by the distribution - she earns 0%"
    );
    assertEq(
        vault.totalAssets(), BOB_STAKE + PREMIUM,
        "the entire premium accrued to the live share register"
    );
    vm.prank(payoutSigner);
    uint256 paid = vault.payout(payoutReceiver, LOSS, pStart);
    assertEq(paid, LOSS, "the full assessed loss was payable");
    assertLt(vault.withdrawAssets(bucket), ALICE_STAKE, "<-- Alice's
    ↪ bucket WAS slashed");
    vm.warp(uint256(pStart) + 2 * PERIOD_DURATION + 1 hours);
    vm.prank(alice);
    uint256 aliceOut = vault.claimWithdraw(bucket);
    uint256 bobOut = vault.previewRedeem(vault.balanceOf(bob));
    uint256 aliceFair = 330_000e6;
    uint256 bobFair = 220_000e6;
    console2.log("--- actual ---");
    console2.log("alice out          :", aliceOut);
    console2.log("bob   out          :", bobOut);
    console2.log("--- pro-rata (fair) ---");
    console2.log("alice fair          :", aliceFair);
    console2.log("bob   fair          :", bobFair);
    console2.log("--- delta ---");
    console2.log("alice shortfall      :", aliceFair - aliceOut);
    console2.log("bob   windfall       :", bobOut - bobFair);
    assertLt(aliceOut, aliceFair, "Alice received less than her pro-rata
    ↪ entitlement");
    assertGt(bobOut, bobFair, "Bob received more");
    assertApproxEqAbs(
        aliceFair - aliceOut, bobOut - bobFair, 2,
        "Alice's shortfall equals Bob's windfall - value moved between
    ↪ cohorts"
    );
    assertApproxEqAbs(
        aliceOut + bobOut, ALICE_STAKE + BOB_STAKE + PREMIUM - LOSS, 2,
        "in == out: the protocol owes exactly what it holds"
    );
}

function testControlNoPendingWithdrawalPremiumSplitsProRata() public {
    _stakeAndEnterPeriodP();
    distributePremium(PREMIUM);

```

```

        uint256 aliceGain = vault.previewRedeem(vault.balanceOf(alice)) -
↪ ALICE_STAKE;
        uint256 bobGain = vault.previewRedeem(vault.balanceOf(bob)) -
↪ BOB_STAKE;
        console2.log("alice gain (no exit) :", aliceGain);
        console2.log("bob   gain (no exit) :", bobGain);
        assertApproxEqAbs(aliceGain, 30_000e6, 2, "Alice's fair share of the
↪ premium is 30,000");
        assertApproxEqAbs(bobGain, 20_000e6, 2, "Bob's is 20,000");
    }
    function testControlNoDistributionSlashIsExactlyProRata() public {
        (uint256 P, uint48 pStart) = _stakeAndEnterPeriodP();
        uint256 bucket = P + 1;
        uint256 aliceShares = vault.balanceOf(alice);
        vm.prank(alice);
        vault.redeem(aliceShares, alice, alice);
        vm.prank(payoutSigner);
        vault.payout(payoutReceiver, LOSS, pStart);
        vm.warp(uint256(pStart) + 2 * PERIOD_DURATION + 1 hours);
        vm.prank(alice);
        uint256 aliceOut = vault.claimWithdraw(bucket);
        uint256 bobOut = vault.previewRedeem(vault.balanceOf(bob));
        console2.log("alice out (no premium):", aliceOut);
        console2.log("bob   out (no premium):", bobOut);
        assertApproxEqAbs(aliceOut, ALICE_STAKE / 2, 2, "Alice loses exactly
↪ her pro-rata 50%");
        assertApproxEqAbs(bobOut, BOB_STAKE / 2, 2, "Bob loses exactly his
↪ pro-rata 50%");
    }
    function testSlashedBucketCanNeverBeRebuiltByAnyDistribution() public {
        (uint256 P, uint48 pStart) = _stakeAndEnterPeriodP();
        uint256 bucket = P + 1;
        uint256 aliceShares = vault.balanceOf(alice);
        vm.prank(alice);
        vault.redeem(aliceShares, alice, alice);
        vm.prank(payoutSigner);
        vault.payout(payoutReceiver, LOSS, pStart);
        uint256 aliceBucketAfterSlash = vault.withdrawAssets(bucket);
        console2.log("alice bucket after slash :", aliceBucketAfterSlash);
        for (uint256 i = 1; i <= 4; ++i) {
            distributePremium(PREMIUM);
            console2.log("--- after distribution", i);
        }
    }

```

```

        console2.log("  alice bucket          :",
↪ vault.withdrawAssets(bucket));
        console2.log("  bob   redemption value :",
↪ vault.previewRedeem(vault.balanceOf(bob)));
        assertEq(
            vault.withdrawAssets(bucket), aliceBucketAfterSlash,
            "<-- Alice's bucket NEVER moves, no matter how many emissions
↪ arrive"
        );
    }
    assertGt(
        vault.previewRedeem(vault.balanceOf(bob)), BOB_STAKE / 2,
        "Bob's redemption value was rebuilt by the emissions"
    );
    assertEq(
        vault.withdrawAssets(bucket), aliceBucketAfterSlash,
        "Alice's was not, and structurally cannot be"
    );
}
}

```

Output

```

$ forge test --match-contract DistributionCohortAsymmetry -vv
Ran 4 tests for
↪ test/Foundry/DistributionCohortAsymmetry.t.sol:DistributionCohortAsymmetry
[PASS] testControlNoDistributionSlashIsExactlyProRata() (gas: 908886)
Logs:
  alice out (no premium): 3000000000000
  bob   out (no premium): 2000000000000
[PASS] testControlNoPendingWithdrawalPremiumSplitsProRata() (gas: 714723)
Logs:
  alice gain (no exit) : 29999999999
  bob   gain (no exit) : 19999999999
[PASS] testPendingWithdrawalBearsTheSlashButEarnsNoneOfThePremium() (gas:
↪ 978629)
Logs:
  --- actual ---
  alice out          : 314285714285
  bob   out          : 235714285715
  --- pro-rata (fair) ---
  alice fair         : 3300000000000

```

```

bob    fair                : 220000000000
--- delta ---
alice shortfall            : 15714285715
bob    windfall            : 15714285715
[PASS] testSlashedBucketCanNeverBeRebuiltByAnyDistribution() (gas: 1054772)
Logs:
  alice bucket after slash : 300000000000
  --- after distribution 1
    alice bucket           : 300000000000
    bob    redemption value : 250000000000
  --- after distribution 2
    alice bucket           : 300000000000
    bob    redemption value : 300000000000
  --- after distribution 3
    alice bucket           : 300000000000
    bob    redemption value : 350000000000
  --- after distribution 4
    alice bucket           : 300000000000
    bob    redemption value : 400000000000
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 18.09ms
↳ (11.38ms CPU time)

```

What the Four Runs Establish

All figures are at 6 decimals: 300000000000 = 300,000 FXRP.

Run	Setup	Result
Control A	Distribution, no pending withdrawal	Premium splits 29,999.999999 / 19,999.999999 — clean 60/40 pro-rata, minus one wei each to the virtual offset.
Control B	Pending withdrawal, no distribution	300,000 / 200,000 — each side loses exactly 50%, perfectly proportional.
The find-ing	Both	314,285.714285 / 235,714.285715 against a fair 330,000 / 220,000.
Test 4	Four emissions after a slash	Alice's bucket: 300,000 five times. Bob: 200,000 → 250,000 → 300,000 → 350,000 → 400,000.

Control A supplies the counterfactual. With no exit in flight, Alice's 60% earns 60% of the premium.

That is the entitlement the finding removes.

Control B forecloses “the slash itself is unfair.” It is not — without a distribution the loss falls exactly pro-rata, 50% on each side. The distribution is the only variable that breaks it.

The finding quantifies the transfer. Alice’s shortfall and Bob’s windfall are both 15,714,285,715 — equal to the wei, which is what makes this a transfer between cohorts rather than a loss of value. The suite asserts that equality, and separately asserts:

```
aliceOut + bobOut == 550,000
               == 1,000,000 + 50,000 - 500,000
```

Test 4 is the documented promise, tested. “Future emission streams continue, rebuilding the redemption value over time.” Four emissions land. Bob’s redemption value doubles. Alice’s bucket returns the same integer on all five reads.

The rebuild is not partial for her — it is structurally impossible, because `withdrawAssets[p] +=` exists at exactly one line in the vault and that line is `_requestWithdraw`.

Dispute / Review History

August 24, 2026 — Report Created

web3pavlou (Whitehat) *August 24, 2026 at 1:08 pm*

Report #90304 created.

August 24, 2026 — Initial Immunefi Review

andrew — All Participants *August 24, 2026 at 1:16 pm*

Hi,

Thanks for the submission,

Your report is currently being reviewed by Immunefi. We will get back to you once a decision has been made regarding your submission.

Best,

The Immunefi team

August 24, 2026 — Escalation

andrew — All Participants *August 24, 2026 at 1:26 pm*

Hello,

Thank you for submitting your report to the project's Audit Competition. We want to inform you that our triage team has reviewed your report and has decided to escalate it to the project for further technical assessment with an addition of full technical analysis breakdown.

Technical correctness of researcher's claims:

The core claim is internally consistent with the code excerpts shown: `payout` computes `activePayableAmount = currentActiveAssets + capturePeriodWithdrawals + nextPeriodWithdrawals` and decrements `withdrawAssets[cp+1] / [cp+2]`, while `distributeRewards/distributeIncentive` only do a `safeTransferFrom` followed by `checkpointTotalAssets()`, never touching `withdrawAssets[*]`.

Since `totalAssets() = super.totalAssets() - pendingWithdrawAssets`, any raw balance increase accrues entirely to live shares, confirming the asymmetric denominators between the loss path and the gain path exactly as described.

PoC check:

The Foundry PoC runs against the real `FirelightVault` and `VaultRewardDistributor` contracts (not mocks) and includes four tests: the main finding, two isolating controls (pro-rata split with no pending withdrawal; exactly pro-rata slash with no distribution), and a fourth showing a slashed bucket never recovers across four emission cycles.

Console output shows Alice receiving 314,285.714285 vs a fair 330,000, with the shortfall (15,714.285715) equal to Bob's windfall to the wei, and total payout conserving 1,000,000 + 50,000 - 500,000 = 550,000, supporting the solvency-preserved/misallocation claim; it does not demonstrate any path by which an attacker or the protocol could exploit this beyond passive reallocation, consistent with the report's own framing that no privileged misuse is involved.

Funds at risk from successful exploitation (in USD):

—

Assessed Impact:

Low severity — contract fails to deliver promised returns without loss of protocol solvency; funds are misallocated between cohorts (exiting stakers under-compensated, staying stakers over-compensated) rather than stolen or frozen.

Assessed Asset:

`contracts/core/FirelightVault.sol` (`payout`, `_requestWithdraw`, `totalAssets`, `claimWithdraw`) and `contracts/core/VaultRewardDistributor.sol` (`distributeRewards`, `distributeIncentive`) — both appear to be core in-scope protocol contracts per the repository paths referenced.

Contextual information:

The report frames this as a non-privileged, unconditional accounting defect (no attacker or mis-used role) that occurs whenever a withdrawal request is pending during any emission distribution and payout event, which the researcher states is the vault's normal steady state given the built-in withdrawal delay; it also distinguishes this from the project's documented "new deposits capture undeserved yield" accepted risk by noting this covers the inverse (exit-side) case.

Please note that this bug bounty program requires Immunefi's triaging. As part of this process, they performed the following checks:

- Impact Assessment — Determine if the impact selected by the bug bounty hunter during the bug reporting process is the one described in the details of the bug report and/or is in-scope of the bug bounty program.
- Asset Assessment — Determine if the asset selected by the bug bounty hunter is the one described in the details of the bug report and/or is in-scope of the bug bounty program.
- Discard out-of-scope reports that passed through Automated Filtering and are obviously unrelated or non-security content, such as, but not limited to:
 - Putting filler or dummy text in required areas.
 - Providing junk information unrelated to a vulnerability in required areas of information.
- Verify that a Proof of Concept (PoC) is attached.
- Assess whether an attached PoC is determined to contain code and the code is determined by the triaging team as appearing to be relevant based on the impact reported and the attack description.
- Ensure that the bug report contains sufficient details within the PoC section and the Bug Description section to properly evaluate the PoC.
- Check whether the report complies with any custom requirements provided by the client in the bug bounty program.

-
- Check whether the report is created by use of any automation tools such as, but not limited to, ChatGPT and Bing's AI.
 - Check whether the report violates any of the out-of-scope rules as stated in the client's bug bounty program.

Please note that the final decision on the validity of the report lies still with the Project.

Please note that the SLA resolution time for High / Critical severity reports is 14 days from date of escalation. For Low / Medium severity reports, SLA resolution time is 7 days.

Resolution of a bug report also includes any applicable bounty payout to the researcher.

If you require any assistance or have any questions, please refer to the submission FAQs or do not hesitate to use the "ask for help" feature to re-subscribe Immunefi with additional queries.

Thank you again for your efforts in making web3 a safer place. We wish you the best of luck with this and your future submissions.

Best regards,

Immunefi

August 24, 2026 — Project Closure

fico23 (Audit Comp | Firelight) *August 24, 2026 at 2:09 pm*

First part: By design. A depositor joins the live pooled vault and assumes its unresolved current/previous-period liabilities; deposit blocking protects against already-declared incidents, not unknown events that are reported retroactively.

Second part, from macro audit: New deposits can capture a pending reward distribution (accepted)

Because rewards are injected by raising the vault redemption value, a deposit placed just before a distribution shares in yield it did not earn, diluting existing stakers for that cycle. It is mitigated by the withdrawal delay and by incident risk, since the new capital cannot exit immediately and remains exposed for the period.

So reward distribution is known.

August 24, 2026 — Researcher Response

web3pavlou (Whitehat) — All Participants *August 24, 2026 at 5:09 pm*

The accepted risk cited is entry-side by its own words — “New deposits can capture a pending reward distribution.”

This report is exit-side: a departing staker loses yield they earned while remaining liable for the loss. Different cohort, opposite direction.

More directly, the mitigation quoted in the closure is this report’s own argument. Macro accepts the entry-side dilution because “the new capital cannot exit immediately and remains exposed for the period” — exposure is the consideration for the yield.

This report demonstrates that capital in a withdrawal bucket has the exposure without the yield: `payout` reaches `withdrawAssets[cp+1]` and `[cp+2]`, while `distributeRewards` touches neither. The symmetry the accepted risk rests on is precisely what is absent.

Test 4 shows a slashed bucket unchanged across four consecutive distributions while the live-share cohort’s redemption value rebuilds. That is not new deposits capturing a distribution.

On the first point: this report makes no claim about deposits and contains no deposit-timing mechanism. Deposit blocking and retroactively reported incidents are not part of it.

Immunefi’s escalation recorded the distinction independently: “it also distinguishes this from the project’s documented ‘new deposits capture undeserved yield’ accepted risk by noting this covers the inverse (exit-side) case.”

August 28, 2026 — Immunefi Follow-Up

YaMi (Immunefi) *August 28, 2026 at 2:27 pm*

YaMi self-subscribed to the report.

August 28, 2026 — Researcher Request for Re-Examination

web3pavlou (Whitehat) — Whitehat & Immunefi *August 28, 2026 at 4:09 pm*

Hi Yami,

Is there any chance someone from Firelight team bothers to check this out? Maybe a second opinion from another member would change the verdict.

Their closing reason cites a prior-audit accepted risk about new deposits capturing a reward distribution. My report is about the opposite case — an exiting staker, not an incoming one — and Immunefi’s own escalation note recorded that distinction before it ever reached the project: “it also distinguishes this from the project’s documented ‘new deposits capture undeserved yield’

accepted risk by noting this covers the inverse (exit-side) case.”

Mediation isn’t a choice here, as this is a low and I’d be paying more than the finding is worth.

I’d die to get a non-trivial answer for this report. It would be great if you could ping the team to reexamine this one, preferably before review is over.

Sincerely, web3pavlou

September 2, 2026 — Follow-Up

web3pavlou (Whitehat) — All Participants *September 2, 2026 at 2:06 pm*

Any updates from the team on this one?